

ENGEE319-26B Laboratory Session 3

Solar Power Monitoring and IoT Dashboard

This lab builds a solar power monitoring system on a Raspberry Pi. The Pi reads panel output through an INA219 sensor, uses a relay to connect or isolate the panel, and logs voltage, current, and power to a ThingSpeak cloud dashboard.

On completion, students will be able to:

- Read voltage, current, and power from an INA219 sensor via I²C.
- Use a Raspberry Pi GPIO pin to control a relay and switch the panel circuit.
- Explain how relay state affects the INA219 readings.
- Configure an XL4015 step-down module and set a safe, stable output voltage.
- Upload telemetry to ThingSpeak and build a live monitoring dashboard.

Equipment

Table 1: Component List

Component	Qty	Description
Raspberry Pi 4 Model B (or 3B+/4B)	1	Runs monitor.py to read the sensor, control the relay, and upload data to ThingSpeak.
Solar panel	1	Converts lamp light into electrical energy. Built-in blocking diode prevents reverse current;
XL4015 step-down module	1	Regulates the variable panel voltage down to a fixed, safe monitoring voltage. Output is set with the on-board potentiometer.
INA219 current/voltage sensor module	1	Communicates over I ² C to provide real-time voltage and current readings to the Pi.
Single-channel relay module (3.3 V logic)	1	Controlled by the Pi GPIO to connect or isolate the solar panel from the rest of the circuit.
Fixed test load (100 Ω , 1 W resistor)	1	Provides a defined load so the INA219 reads a meaningful current. Replaces the battery used in Lab 4.
Breadboard and jumper wires	1 set	Routes power, ground, and data/control signals between components.

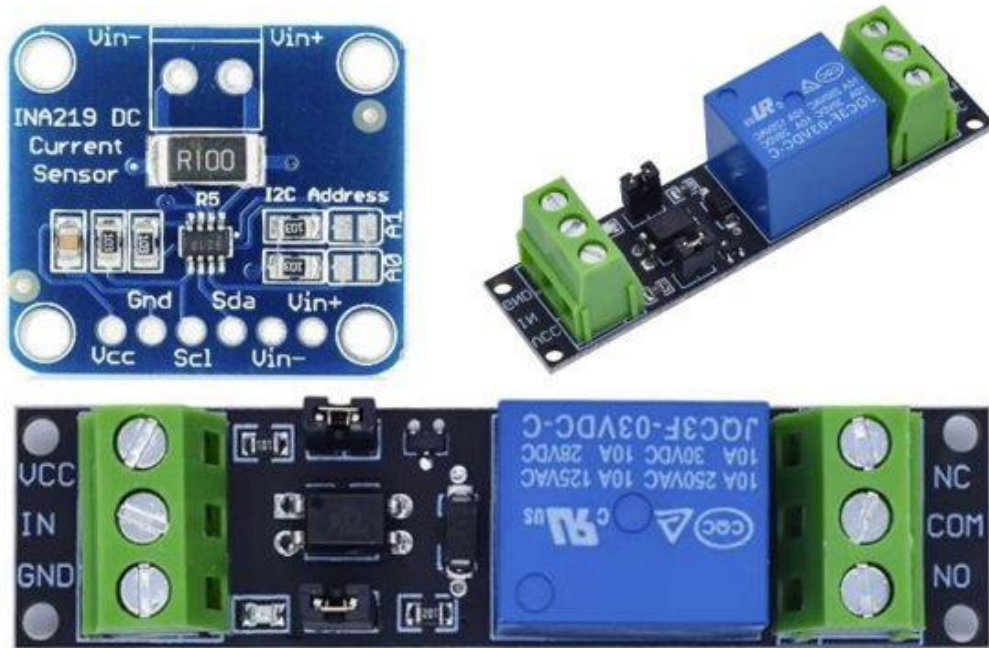


Figure 1: INA219 sensor module (left) and relay module (right).

Section 0: Environment Setup

0.1 Create virtual environment

```
python -m venv pythonenv
source pythonenv/bin/activate
```

0.2 Install packages

```
sudo apt install -y swig liblgpio-dev
pip install --upgrade pip
pip install smbus2 pi-ina219 rpi-lgpio requests
```

Package	Purpose
smbus2	Pure-Python I ² C/SMBus driver (replaces smbus).
pi-ina219	High-level INA219 driver; uses smbus2 internally.
rpi-lgpio	RPi.GPIO drop-in replacement for kernel 6.x.
requests	HTTP client for ThingSpeak uploads.

0.3 Enable and verify I²C

```
sudo raspi-config      # Interface Options -> I2C -> Enable
i2cdetect -y 1         # 0x40 should appear once INA219 is wired
```

Section 1: Hardware Wiring

1.1 Circuit architecture

The monitoring circuit follows this component order:

```
Solar Panel (+) -> Relay COM -> Relay NO -> XL4015 IN+ ->  
XL4015 OUT+ -> INA219 (+In -> 0.1 ohm shunt -> -In) ->  
Test load (100 ohm) -> System GND -> Solar Panel (-)
```

Placing the relay first lets the Pi connect or isolate the panel under software control:

- Relay open: the panel is isolated from the rest of the circuit. The INA219 reads close to zero current.
- Relay closed: current flows Solar -> XL4015 (regulates voltage) -> INA219 shunt -> test load. The INA219 measures the circuit voltage and current.

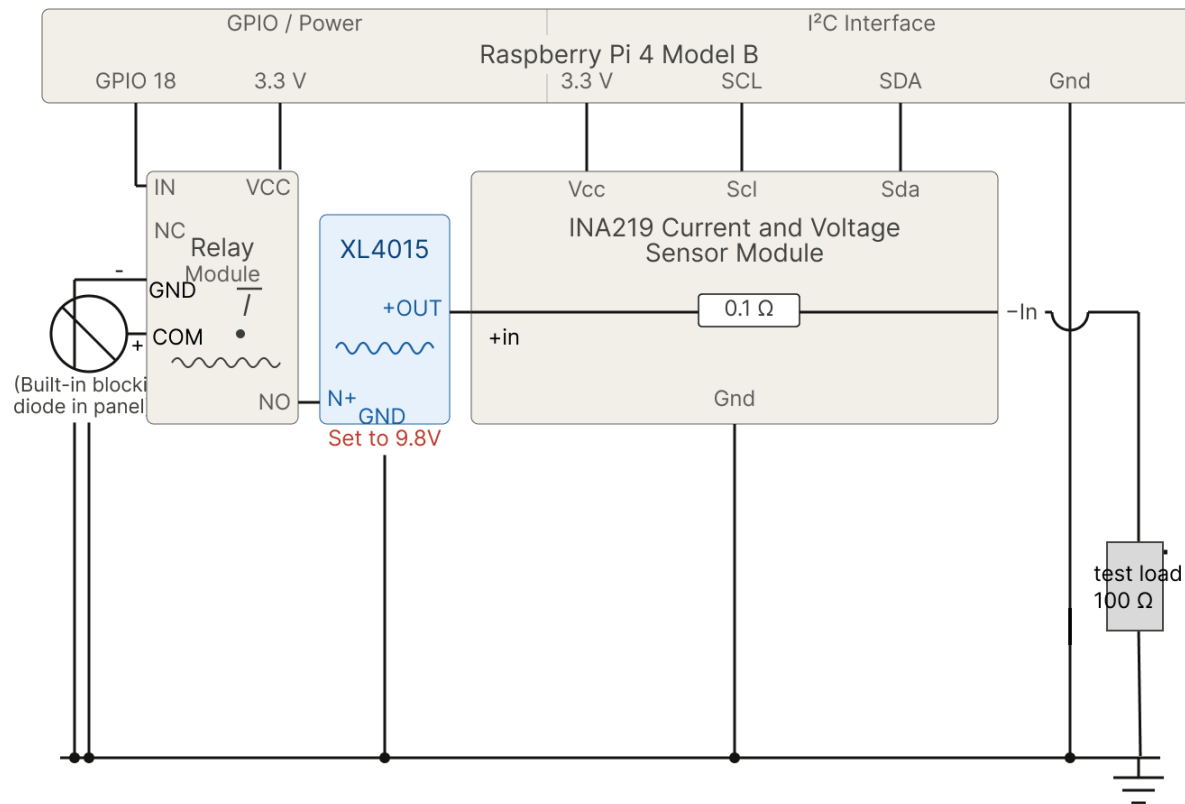


Figure 2: Lab 3 monitoring circuit. Solar -> Relay -> XL4015 -> INA219 -> 100 Ω test load. No battery, no external Schottky diode required.

1.2 Pin reference

Pin reference: <https://pinout.xyz>

Signal	Pi Physical Pin	Pi BCM GPIO	Connects to	Wire colour (suggested)
GPIO (relay control)	Pin 12	GPIO 18	Relay IN	Yellow
3.3 V (relay)	Pin 1	—	Relay VCC	Red
GND (relay)	Pin 6	—	Relay GND	Black
3.3 V (sensor)	Pin 17	—	INA219 Vcc	Red
I ² C clock	Pin 5	GPIO 3	INA219 Scl	Blue
I ² C data	Pin 3	GPIO 2	INA219 Sda	Green
GND (sensor)	Pin 9	—	INA219 Gnd	Black
GND (system)	Pin 14 or 20	—	Solar (–), XL4015 GND, test load (–)	Black

1.3 Step-by-step wiring

- Step 1. Activate the virtual environment.
- Step 2. Connect the relay control pins (table above).
- Step 3. Connect the INA219 I²C pins.
- Step 4. Verify relay and I²C:

```
python3 -c "import RPi.GPIO as G,time; G.setmode(G.BCM); G.setup(18,G.OUT); \
G.output(18,1); time.sleep(1); G.output(18,0); G.cleanup()"
i2cdetect -y 1 # 0x40 should appear
```

- Step 5. Connect Solar Panel (+) to Relay COM. Connect Solar Panel (–) to system GND.
- Step 6. Connect Relay NO -> XL4015 IN+. Connect XL4015 IN– to system GND.
- Step 7. Set the XL4015 output (procedure below), then connect XL4015 OUT+ -> INA219 +In and the test load to INA219 –In.

✓ CHECKPOINT 1 — Wiring verified

- Relay clicks when GPIO 18 is toggled from the command line.
- i2cdetect -y 1 shows address 0x40.
- XL4015 output is stable and noted with a multimeter.

Section 2: Power Monitoring

2.1 I²C architecture

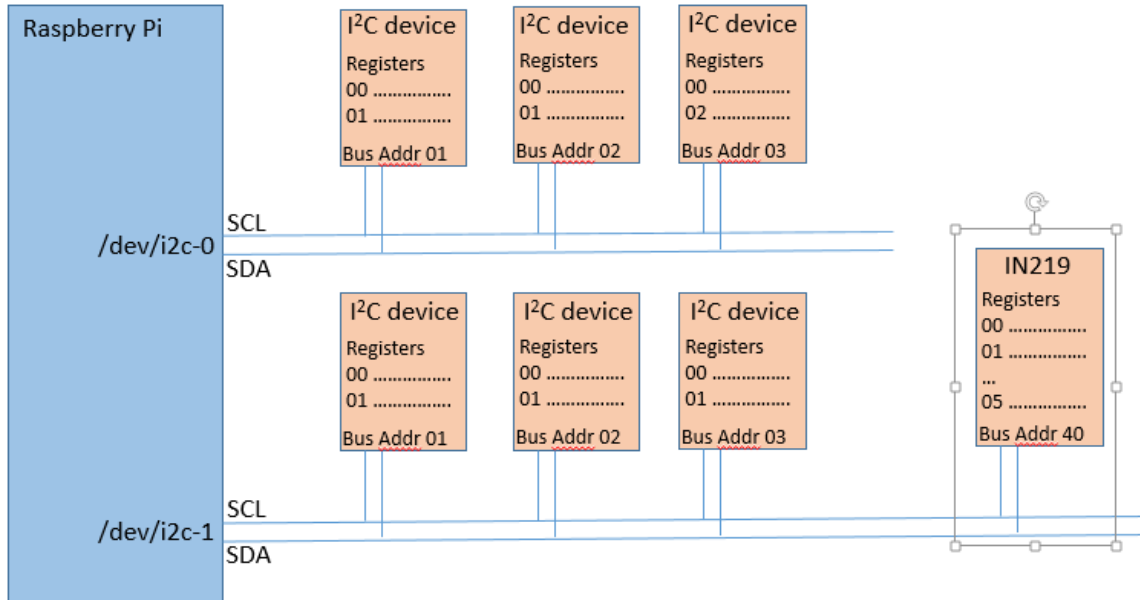


Figure 3: I²C bus. The INA219 is at address 0x40 on /dev/i2c-1.

2.2 monitor.py

Download monitor.py from Moodle (copying from a PDF can introduce invisible characters that cause SyntaxError). With the relay open or closed, run the script and vary the lamp distance.

```
# monitor.py - Lab 3: solar monitoring + ThingSpeak upload
# Platform: Raspberry Pi 4B, OS Bookworm, Python 3.11, kernel 6.1
# Note: download from Moodle to avoid PDF line-wrap errors.
from ina219 import INA219
import RPi.GPIO as GPIO
from time import sleep, time
import requests

RELAY_PIN    = 18
API_KEY      = "YOUR_WRITE_API_KEY_HERE"
TS_URL       = "https://api.thingspeak.com/update"
UPLOAD_INTVL = 15      # s: ThingSpeak free-tier minimum

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY_PIN, GPIO.OUT)

# Change RELAY_STATE to complete the Observation Table.
# GPIO.HIGH = Relay Closed  GPIO.LOW = Relay Open
RELAY_STATE = GPIO.HIGH
GPIO.output(RELAY_PIN, RELAY_STATE)
state_str = "CLOSED" if RELAY_STATE == GPIO.HIGH else "OPEN"
```

```

ina = INA219(shunt_ohms=0.1, max_expected_amps=3.0, busnum=1, address=0x40)
ina.reset()
ina.configure(voltage_range=ina.RANGE_32V, gain=ina.GAIN_AUTO,
              bus_adc=ina.ADC_4SAMP, shunt_adc=ina.ADC_4SAMP)

last_upload = 0.0
def upload(v, i, p):
    global last_upload
    if time() - last_upload < UPLOAD_INTVL: return
    params = {"api_key": API_KEY, "field1": round(v,3),
              "field2": round(i,1), "field3": round(p,4)}
    try:
        r = requests.get(TS_URL, params=params, timeout=8)
        if r.ok: last_upload = time()
    except requests.RequestException: pass

print(f"Monitoring. Relay: {state_str}. Ctrl+C to stop.")
try:
    while True:
        v = ina.voltage()
        i = ina.current()
        p = v * (i / 1000.0)
        print(f"Relay:{state_str} | V={v:.3f}V | I={i:.1f}mA | P={p:.4f}W")
        upload(v, i, p)
        sleep(0.5)
except KeyboardInterrupt:
    GPIO.output(RELAY_PIN, GPIO.LOW)
    GPIO.cleanup()
    print("Relay opened safely.")

```

monitor.py both prints readings to the terminal and uploads them to ThingSpeak every 15 seconds. Set your Write API Key in API_KEY before running (see Section 3).

2.3 Observations

Change RELAY_STATE in the code, re-run, and record the readings.

Condition	Relay	V (V)	I (mA)	P (W)
No light, covered	Closed			
Lamp, any distance	Open			
Lamp 60 cm	Closed			
Lamp 20 cm	Closed			

Discussion Question: With the relay open, why does the INA219 read almost no current even when the lamp is shining on the panel?

Section 3: ThingSpeak Dashboard

The `upload()` function is already in `monitor.py`. This section covers channel setup. The same channel is reused in Lab 4.

3.1 Create channel

Sign in at thingspeak.mathworks.com with your MathWorks account. Go to Channels -> My Channels -> New Channel.

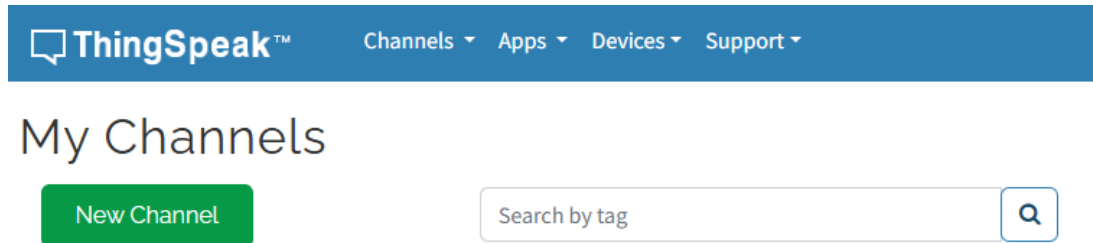


Figure 4: ThingSpeak My Channels page.

Field	Name	monitor.py var	Description
Field 1	Voltage (V)	field1	XL4015 output / battery terminal voltage
Field 2	Current (mA)	field2	Charging current through INA219 shunt
Field 3	Power (W)	field3	Panel output power
Field 4	Status	field4	2 = Full, 1 = Charging, 0 = Standby

New Channel

Name	ENGEE 319 Solar Data	
Description	IoT-based solar battery charging controller	
Field 1	Voltage (V)	☒
Field 2	Current (mA)	☒
Field 3	Power (W)	☒
Field 4	Status	☒
Field 5		☐

Figure 5: New Channel form. Rename fields as shown above.

3.2 API key

On the API Keys tab, copy the Write API Key into the API_KEY variable in monitor.py.

ENGEE 319 Solar Data

Channel ID: **3382042**

Author: mwa0000040803036

Access: Private

IoT-based solar battery charging controller

[Private View](#) [Public View](#) [Channel Settings](#) [Sharing](#) [API Keys](#) [Data Import / Export](#)

Write API Key

Key

[Generate New Write API Key](#)

Read API Keys

Key

Note

[Save Note](#)

[Delete API Key](#)

Help

API keys enable you to write data to a channel or read data from a private channel. API keys are auto-generated when you create a new channel.

API Keys Settings

- **Write API Key:** Use this key to write data to a channel. If you feel your key has been compromised, click [Generate New Write API Key](#).
- **Read API Keys:** Use this key to allow other people to view your private channel feeds and charts. Click [Generate New Read API Key](#) to generate an additional read key for the channel.
- **Note:** Use this field to enter information about channel read keys. For example, add notes to keep track of users with access to your channel.

API Requests

[Write a Channel Feed](#)

GET https://api.thingspeak.com/update?api_key=AJCDCWE3QYHLT9Z7&field1=0

[Read a Channel Feed](#)

Figure 6: API Keys tab. The Write API Key is shown at the top.

3.3 Dashboard

After 30 seconds of uploads, add visualisations:

- Line chart for Field 3 (Power).
- Dual-axis chart: Field 1 (Voltage) and Field 2 (Current).



Figure 7: Solar Data Acquisition and IoT Demonstration Dashboard.

✓ CHECKPOINT 2 — Dashboard live

- Power chart updates as lamp intensity is varied.
- Voltage and current both appear on the dual-axis chart.
- You can explain the 15 s upload limit and how it is enforced in code.

Appendix A: Reference Links

INA219 Python library: github.com/chrisb2/pi_ina219

INA219 datasheet (TI): ti.com/lit/ds/symlink/ina219.pdf

XL4015: https://www.aliexpress.com/item/1005012000581793.html?spm=a2g0o.order_list.order_list_main.50.45a91802iQcl04

ThingSpeak docs: mathworks.com/help/thingspeak

Pi GPIO pinout: pinout.xyz

rpi-lgpio (PyPI): pypi.org/project/rpi-lgpio